FoldPulse

Every Fold. Every Beat.

PHASE 3 · ARCHITECTURE & PROJECT FOUNDATION

Architecture Report & Build Verification

Version	1.0
Date	9 November 2026
Prepared for	Founder & Product Owner
Prepared by	FoldPulse product team (12 agent roles)
Status	Submitted for Founder approval
Recommendation	**APPROVE** (see open items)

Confidential — prepared for the FoldPulse Founder. Build results are from the project's CI.

Contents

1 Executive summary & recommendation 2

2 Architecture 4

3 Database design 8

4 Project foundation: files & key code 11

5 Build & test verification 13

6 Agent reviews & resolutions 15

7 Technical decisions 17

8 Decisions requested & next steps 19

1 Executive summary & recommendation

PHASE 3 RECOMMENDATION

APPROVE

The foundation builds and all 43 automated tests pass in CI. The High and Medium review findings are fixed.

What this phase delivers

FoldPulse now has a **real, buildable iOS project**, verified independently by continuous integration:

- **Architecture:** a pure-Swift domain package (`FoldPulseCore`), an app target, a widget extension and shared code, with documented layers, data flow and concurrency model (Swift 6, complete strict concurrency).
- **Data layer:** a versioned SwiftData schema with a migration plan from day one, a single-writer `EventStore` actor, tombstones instead of deletes, duplicate-id protection, and explicit file protection that works while the phone is locked.
- **Design system in code:** colour tokens with the measured contrast from Phase 2, typography, and seven components (hero counter, stat tile, source badge, milestone bar, "Counted for you" tile, 7-day chart, pulse line).
- **App shell:** four tabs; Today runs on real data; a basic History; Insights and Settings placeholders. A debug-only "Insert sample week" lets you review the design on your iPhone Duo.
- **Widgets:** Home small (StandBy), Lock Screen circular and rectangular. They read a small snapshot file, never the database, and follow the midnight, staleness and privacy rules.
- **Tests and CI:** 43 automated tests, all passing on Linux and macOS (simulator), plus a privacy guard that fails the build if networking code appears.

How it was verified

Check	Result
Core logic tests on Linux and macOS	29 / 29 pass
App unit tests (SwiftData, snapshot service)	13 / 13 pass
UI test (launch, all tabs)	1 / 1 pass
Privacy guard (no networking code)	Pass
Build warnings	One real warning (widget version) found and fixed

CI found three problems along the way. The cause and fix for each are in Chapter 5. None was a compile error in the app code.

Independent reviews

Two reviewer agents (Engineer + Performance; Privacy + QA + Independent Reviewer) found **2 High and 11 Medium issues**. Examples: the widget could stay hours out of date; statistics were recomputed on the main thread; editing a time left the event on the wrong day; duplicate IDs silently overwrote rows; the docs overclaimed. **All High and Medium items were fixed before this report** (Chapter 6).

Recommendation

Approve Phase 3. Open items, recorded and not hidden:

1. **CI uses the iOS 26.5 SDK** because GitHub's runners don't have Xcode 27.1 yet. The 27.1 build on your Mac is the remaining check, and it becomes mandatory in Phase 4, when hinge code needs the 27.1 SDK.
2. **R-29:** App Groups on a device may require the paid Apple Developer Program.

3. **R-28 / I-24** (carried from Phase 2): background automation not yet verified; inner tab bar when half-folded not yet checked.

2 Architecture

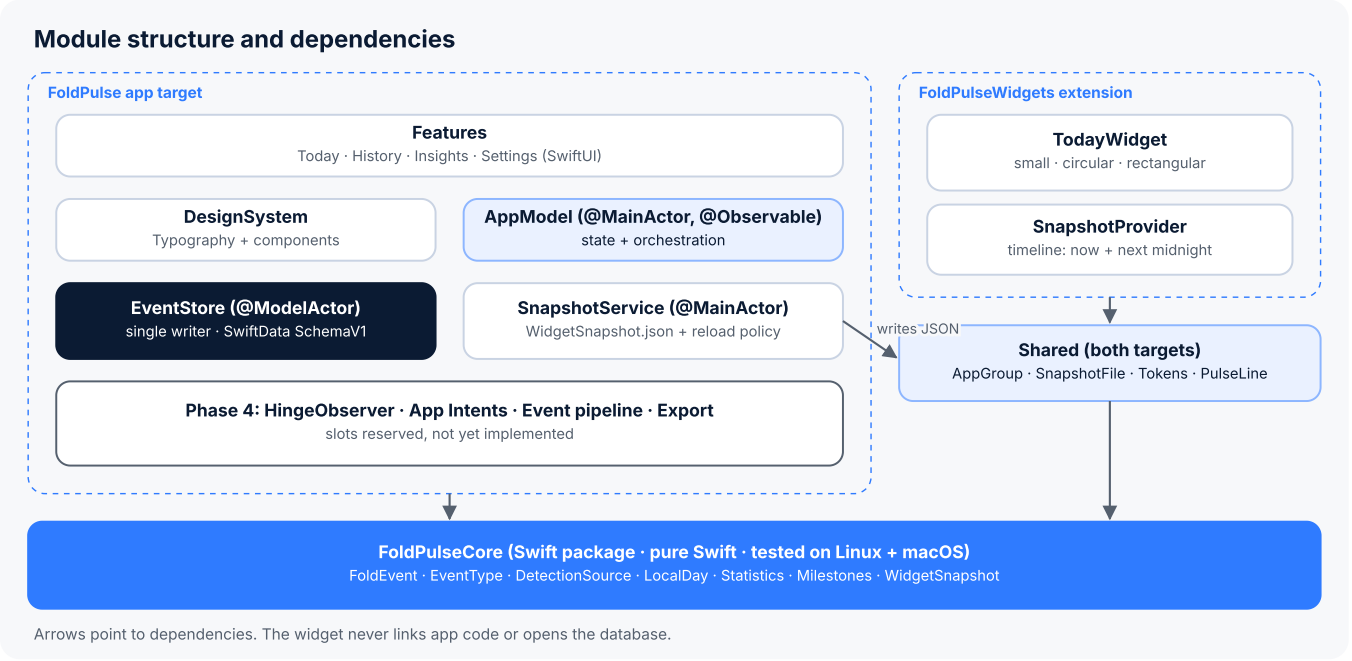


Figure 1 — Module structure and dependencies

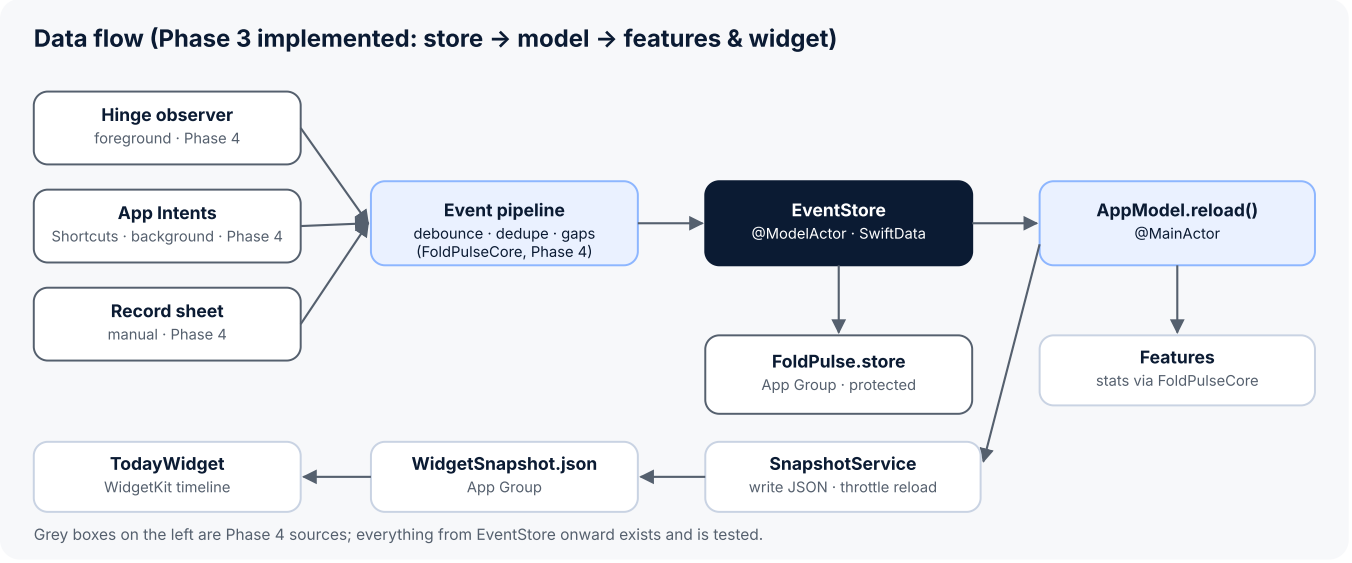


Figure 2 — Data flow

1. Principles

1. **The event log is the source of truth.** Every number is computed from it, and nothing derived is stored as authoritative (T-1).
2. **Pure domain logic lives in its own package** (`FoldPulseCore`). It has no UIKit, SwiftUI or SwiftData, so it is deterministic and testable on Linux and macOS.
3. **One database writer.** All persistence goes through the `EventStore` actor (T-2).
4. **Local only.** There is no networking code; CloudKit is explicitly off (T-3).
5. **Widgets never open the database.** They read a small JSON snapshot (WIDGET_DESIGN §2).
6. **System components first.** `TabView` , `List` , `Chart` and `Gauge` let the system handle iPhone Duo layouts, Liquid Glass and accessibility.
7. **Swift 6 with complete strict concurrency** in every target.

2. Module structure

```
FoldPulse (repo)
├── project.yml                  XcodeGen spec → FoldPulse.xcodeproj (generated, not committed)
├── Packages/FoldPulseCore/      Swift package — domain logic (pure Swift)
│   ├── Sources/FoldPulseCore/   FoldEvent, EventKinds, LocalDay, Statistics, Milestones, WidgetSnapshot
│   └── Tests/                   XCTest — runs on Linux and macOS
├── Shared/                      compiled into BOTH app and widget
│   ├── Storage/                 AppGroup (paths), SnapshotFile (read/write + file protection)
│   └── DesignSystem/            Tokens (colours, spacing, radii), PulseLine
├── FoldPulse/                   app target
│   ├── App/                     FoldPulseApp (entry), RootView (tabs), AppModel (state)
│   ├── Core/Persistence/        SchemaV1 (@Model), mapping, PersistenceController, EventStore (@ModelActor)
│   ├── Core/Services/           SnapshotService (widget snapshot + reload policy)
│   ├── DesignSystem/            Typography, components (HeroCounter, StatTile, SourceBadge, …)
│   ├── Features/                Today · History · Insights · Settings
│   └── Resources/               Assets (icon, accent), PrivacyInfo.xcprivacy, entitlements
├── FoldPulseWidgets/            widget extension (TodayWidget: small, circular, rectangular)
├── FoldPulseTests/              app unit tests (in-memory SwiftData)
├── FoldPulseUITests/            UI smoke tests
├── scripts/setup_mac.sh         one-line developer setup
└── .github/workflows/ci.yml     CI: Linux core tests + macOS build & tests
```

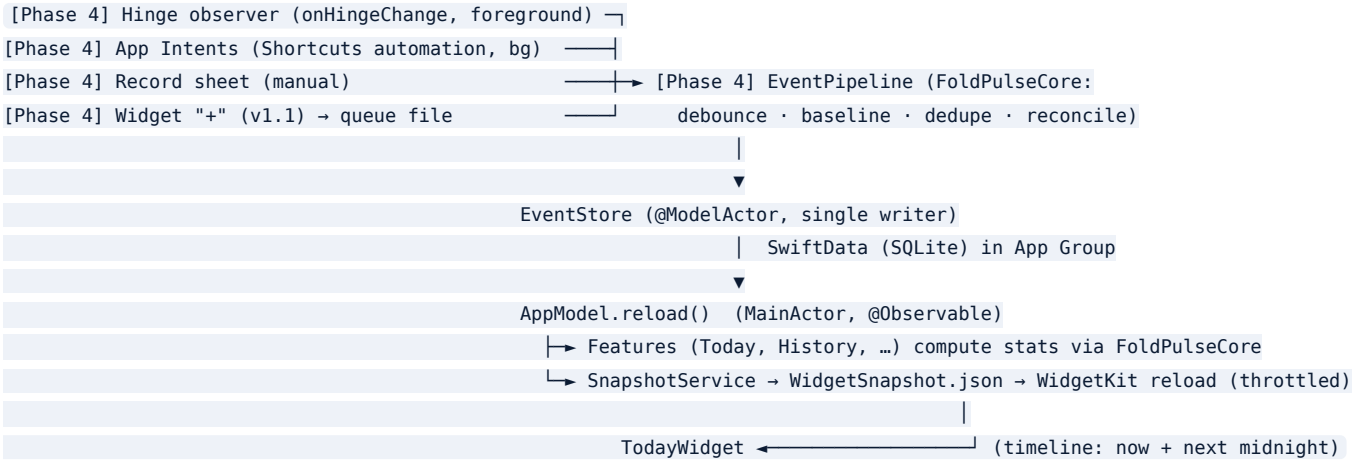
Dependency rule: **Features → AppModel → EventStore / SnapshotService → FoldPulseCore**. Shared depends only on FoldPulseCore and Apple frameworks. The widget depends on Shared + FoldPulseCore, never on app code.

3. Targets and capabilities

Target	Type	Bundle ID	Capabilities
FoldPulse	iOS app (iPhone)	`io.github.odthientho.foldpulse`	App Group `group.io.github.odthientho.foldpulse`
FoldPulseWidgets	WidgetKit extension	`…foldpulse.widgets`	Same App Group
FoldPulseTests	Unit tests	`…foldpulse.tests`	—
FoldPulseUITests	UI tests	`…foldpulse.uitests`	—

Deployment target **iOS 27.1** (D-3). Mac ("Designed for iPhone") and visionOS availability are **off** (R-06). Category: Utilities (R-27).

4. Data flow



The diagram for the PDF is Figure 2 of the Phase 3 report.

5. Concurrency model

Component	Isolation	Notes
SwiftUI views, `AppModel`, `SnapshotService`	`@MainActor`	`AppModel` is `@Observable` and injected with `.environment`
`EventStore`	`@ModelActor` (its own serial executor), **created on a detached task** so its context isn't bound to the main thread	Exchanges only `Sendable` values (`FoldEvent`, `LoadedState`); computes `DashboardStats` itself, so statistics never run on the main thread
`FoldPulseCore`	Non-isolated, pure value types	All types `Sendable`
Widget provider	WidgetKit-managed	Reads a file; no shared mutable state

6. Persistence

See **DATABASE_DESIGN.md**. In summary:

- SwiftData `SchemaV1.StoredEvent` with a `SchemaMigrationPlan` from day one.
- The store file lives in the App Group container.
- Rows are tombstoned instead of deleted.
- File protection is `completeUntilFirstUserAuthentication`, **set explicitly** on the store directory, so background writes work while the phone is locked.
- Release builds never fall back to a private location if the App Group is missing; they show a clear error instead.

7. Widget pipeline

- `SnapshotService.publish` writes the precomputed `WidgetSnapshot` atomically and calls `WidgetCenter.reloadTimelines(ofKind:)`. That reload is immediate when the app is active. From the background it is coalesced to at most one per 5 minutes; the last-reload time is kept in the **App Group defaults**, so the throttle holds across short-lived intent processes.
- The `TodayWidget` timeline has up to three entries: now; the moment the data turns **stale** (shows "as of HH:mm"); and the **next local midnight** ("today" shows "—" until refreshed). The widget also re-reads the snapshot **every 30 minutes**, so a reload the app throttled is never lost.

8. Testing strategy

Layer	Framework	Where it runs	Phase 3 coverage
Domain (`FoldPulseCore`)	XCTest via `swift test`	**CI Linux + CI macOS**	LocalDay (DST, travel, leap year, ranges), totals (gaps, tombstones, duplicates), averages (full days, zero days, window cap), coverage, busiest day, milestones (fire once), snapshot (encoding, schema guard, midnight, staleness)
Persistence & services	XCTest, in-memory SwiftData	CI macOS (simulator)	Field-preserving round trip, ordering, update, tombstone, not-found, delete-all, snapshot write and reload throttling
UI	XCUITest	CI macOS (simulator)	Launch into Today (empty state), all four tabs reachable
On device	Manual	Founder's Mac + iPhone Duo	Sample week via the debug-only Settings action

9. Build and run

- **Mac:** `curl -fsSL https://raw.githubusercontent.com/odthientho/FoldPulse/main/scripts/setup_mac.sh | bash`, then choose a Team for both targets and press Run.
- **CI:** every push to `main` runs `.github/workflows/ci.yml`.
  - On Linux: core tests in the `swift:6.1` container.
  - On macOS: the newest Xcode; core tests, then XcodeGen, then `xcodebuild test` on a simulator.

- If the runner's iOS SDK is older than 27.1, CI lowers the deployment target in `project.yml` **for CI only** and says so in the job summary. GitHub's runners currently have the **iOS 26.5 SDK**. Phase 3 code uses no iOS 27-only API, so build and tests are meaningful there. From Phase 4, hinge code needs the 27.1 SDK; it will be compiled conditionally or verified on the Founder's Mac.
- A **privacy guard** fails the build if networking symbols appear in shipping Swift code (PRIVACY_REVIEW P-1).

10. Phase 4 extension points

Feature	Where it goes
Event pipeline (debounce, baseline, symmetric dedupe ±10 s, gap reconciliation)	`FoldPulseCore/Pipeline` — pure, unit-tested on Linux
Hinge observer	`FoldPulse/Core/Services/HingeObserver` (one per process, foreground scene only)
App Intents (Record Fold/Unfold, `.background`)	App target, writing through `EventStore`
Manual record / Undo, History edit/delete, Insights, Coverage, Setup + self-test, Export, Reset, Milestones, Onboarding	`Features/*`

11. Known limitations and open items

ID	Item	Plan
I-24	Inner-display tab bar position when partially folded	Check in Xcode 27.1 Device Hub on the Founder's Mac (CI runners may not have the iPhone Duo simulator)
R-29	**App Groups may need a paid Apple Developer Program membership** for device installs (free personal teams have restricted capabilities)	If signing fails on the Founder's Mac, either join the Program ($99/year, also required for App Store release) or temporarily remove the App Group (widgets then show the empty state)
R-01	No iPhone Duo in CI	On-device checks stay with the Founder

3 Database design

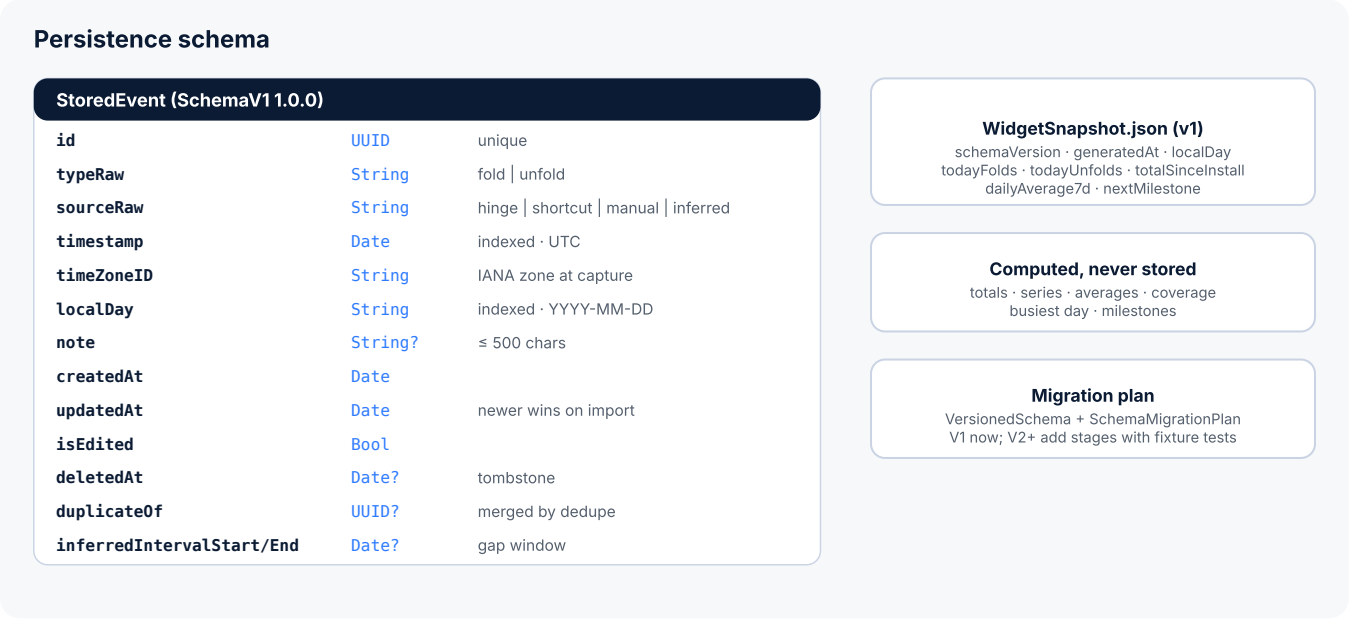


Figure 3 — Persistence schema

1. Technology decision (final)

SwiftData, stored in the App Group container (DECISIONS T-2, confirmed T-6).

Criterion	SwiftData	Core Data
Minimum iOS (27.1)	Fully available, mature since iOS 18 (`#Index` , `#Unique` , history)	Available
Migrations	`VersionedSchema` + `SchemaMigrationPlan`	Model versions + mapping models
Swift 6 concurrency	`@ModelActor` gives an isolated writer	Manual context/queue discipline
Widget compatibility	Not needed: widgets read a JSON snapshot	Same
Complexity	Low	Higher

Core Data stays the fallback if SwiftData blocks something in Phase 4. `EventStore` hides the choice from the rest of the app.

2. Entity: `StoredEvent` (SchemaV1, version 1.0.0)

Field	Type	Rules
`id`	UUID	**Unique.** `EventStore.add` **rejects** an existing id (`StoreError.duplicateID`) because SwiftData's `.unique` would silently overwrite the row. Never changes; the import merge key
`typeRaw`	String	`fold` / `unfold` (`EventType`)
`sourceRaw`	String	`hinge` / `shortcut` / `manual` / `inferred` (`DetectionSource`)
`timestamp`	Date	UTC instant of the event. **Indexed**
`timeZoneID`	String	IANA zone at capture, e.g. `Europe/London` . Kept when the time is edited
`localDay`	String	`YYYY-MM-DD` in the capture zone. **Indexed**; the key for daily buckets. Recomputed (in the capture zone) when the time is edited (`FoldEvent.setTimestamp`)
`note`	String?	≤ 500 characters (enforced in Phase 4 UI)
`createdAt`	Date	First insert
`updatedAt`	Date	Bumped on every change; newer wins on import

Field	Type	Rules
`isEdited`	Bool	The user changed an automatically recorded event
`deletedAt`	Date?	**Tombstone.** Rows are never physically deleted except by Reset
`duplicateOf`	UUID?	Set when merged into another event by de-duplication (Phase 4)
`inferredIntervalStart/End`	Date?	For gaps: the window in which the missed event happened

Rows whose `typeRaw` or `sourceRaw` the app doesn't recognise (written by a newer version) are **skipped, not guessed** (`StoredEvent.domainEvent`).

3. Derived data: computed, not stored

Metric	Computed by	Notes
Totals, today, per-day series	`Statistics.total` / `.series`	Excludes tombstones, duplicates and (by default) gaps
Daily average	`Statistics.dailyAverage`	Last 7 **full** days before today, zero days included, never before the first tracked day
Coverage	`Statistics.coverage`	Share of **recorded** folds captured without tapping, plus gap count
Milestones	`Milestones`	Celebrated once; celebrated set persisted in Phase 4 (`UserDefaults`)
Widget snapshot	`WidgetSnapshot.make`	Written to JSON after each change

Performance: a heavy user (~150 folds a day ≈ 300 events) creates ~110,000 rows a year, about 20–30 MB. Statistics are computed **once per reload inside the `EventStore` actor** (`loadState` → `DashboardStats`), off the main thread; views only read the result. Loading reuses the stored `localDay` and caches time zones, so there is no per-row calendar work. Tombstones are filtered in the fetch predicate. If Phase 5 profiling shows a need, per-day aggregate queries on the `localDay` index replace loading every row.

4. Files and protection

File	Location	Protection	Writer	Readers
`FoldPulse.store` (+ `-wal`, `-shm`)	App Group container, `Store/` directory	`completeUntilFirstUserAuthentication`, **set explicitly on the directory** (new files inherit it). Verified on device in Phase 5	App (`EventStore`)	App
`WidgetSnapshot.json`	App Group container	`completeUntilFirstUserAuthentication` (set explicitly on write)	App (`SnapshotService`)	Widget
`pending-events` queue (v1.1, widget button)	App Group	Same	Widget	App

No silent fallback: release builds refuse to start (clear message) if the App Group is unavailable, rather than writing data to a private location that would be stranded later. Debug/CI builds fall back to Application Support.

Backups: App Group data is included in the user's own iCloud or Finder device backups (PRIVACY_REVIEW P-7). Deleting the app deletes the data.

5. Snapshot JSON (schema version 1)

```
{
  "dailyAverage7d": 72,
  "generatedAt": 1794233040,
  "localDay": "2026-11-09",
  "nextMilestone": 5000,
  "schemaVersion": 1,
```

```
  "todayFolds": 64,
  "todayUnfolds": 62,
  "totalSinceInstall": 1284
}
```

Readers reject a newer `schemaVersion` and fall back to the empty state instead of misreading it.

6. Migration strategy

1. Never edit `SchemaV1` after release. Add `SchemaV2` and a `MigrationStage` (lightweight where possible, custom with tests otherwise).
2. Every migration gets a unit test that opens a V(n−1) store fixture and checks that every row survives.
3. Snapshot JSON changes bump `schemaVersion`. Widgets ship in the same bundle, so they update together.

7. Integrity rules

- IDs are unique: enforced by `EventStore.add`, which rejects duplicates (tested).
- Fold and unfold alternation is **checked, not enforced**: violations become `inferred` gaps (Phase 4 reconciliation). Recorded data is never rewritten.
- Edits keep the original `timeZoneID` and set `isEdited`.
- Reset (Phase 4, Settings) will remove all rows, the snapshot and settings, then reload widgets. In Phase 3, `deleteAll` exists and is exposed only in debug builds.

4 Project foundation: files & key code

Every file below is in the repository (`odthientho/FoldPulse`, branch `main`). The Xcode project is generated from `project.yml` by XcodeGen.

```
.github/workflows/ci.yml
FoldPulse/App/AppModel.swift
FoldPulse/App/FoldPulseApp.swift
FoldPulse/App/RootView.swift
FoldPulse/Core/Persistence/EventStore.swift
FoldPulse/Core/Persistence/PersistenceController.swift
FoldPulse/Core/Persistence/Schema.swift
FoldPulse/Core/Persistence/StoredEvent+Mapping.swift
FoldPulse/Core/Services/SnapshotService.swift
FoldPulse/DesignSystem/Components/CoverageTile.swift
FoldPulse/DesignSystem/Components/HeroCounter.swift
FoldPulse/DesignSystem/Components/MilestoneBar.swift
FoldPulse/DesignSystem/Components/SourceBadge.swift
FoldPulse/DesignSystem/Components/StatTile.swift
FoldPulse/DesignSystem/Components/WeekChart.swift
FoldPulse/DesignSystem/Typography.swift
FoldPulse/Features/History/HistoryView.swift
FoldPulse/Features/Insights/InsightsView.swift
FoldPulse/Features/Settings/SettingsView.swift
FoldPulse/Features/Today/TodayView.swift
FoldPulse/Resources/PrivacyInfo.xcprivacy
FoldPulseTests/EventStoreTests.swift
FoldPulseTests/SnapshotServiceTests.swift
FoldPulseUITests/LaunchTests.swift
FoldPulseWidgets/FoldPulseWidgets.swift
FoldPulseWidgets/PrivacyInfo.xcprivacy
FoldPulseWidgets/TodayWidget.swift
Packages/FoldPulseCore/Package.swift
Packages/FoldPulseCore/Sources/FoldPulseCore/DashboardStats.swift
Packages/FoldPulseCore/Sources/FoldPulseCore/EventKinds.swift
Packages/FoldPulseCore/Sources/FoldPulseCore/FoldEvent.swift
Packages/FoldPulseCore/Sources/FoldPulseCore/LocalDay.swift
Packages/FoldPulseCore/Sources/FoldPulseCore/Milestones.swift
Packages/FoldPulseCore/Sources/FoldPulseCore/Statistics.swift
Packages/FoldPulseCore/Sources/FoldPulseCore/WidgetSnapshot.swift
Packages/FoldPulseCore/Tests/FoldPulseCoreTests/Fixtures.swift
Packages/FoldPulseCore/Tests/FoldPulseCoreTests/LocalDayTests.swift
Packages/FoldPulseCore/Tests/FoldPulseCoreTests/MilestonesTests.swift
Packages/FoldPulseCore/Tests/FoldPulseCoreTests/ReviewFixTests.swift
Packages/FoldPulseCore/Tests/FoldPulseCoreTests/StatisticsTests.swift
Packages/FoldPulseCore/Tests/FoldPulseCoreTests/WidgetSnapshotTests.swift
Shared/DesignSystem/PulseLine.swift
Shared/DesignSystem/Tokens.swift
Shared/Storage/AppGroup.swift
Shared/Storage/SnapshotFile.swift
project.yml
scripts/setup_mac.sh
```

FoldPulse/Core/Persistence/EventStore.swift

```swift
@ModelActor
actor EventStore {
    enum StoreError: Error, Equatable {
        case notFound(UUID)
        /// SwiftData's `.unique` would silently overwrite; we reject instead (Phase 3 review).
        case duplicateID(UUID)
    }

    /// Inserts a new event. Rejects an id that already exists.
    func add(_ event: FoldEvent) throws {
        if try row(event.id) != nil { throw StoreError.duplicateID(event.id) }
        modelContext.insert(StoredEvent(event))
        try modelContext.save()
    }

    /// Inserts many events with a single save (rejects any existing id).
    func add(contentsOf events: [FoldEvent]) throws {
        for event in events {
            if try row(event.id) != nil { throw StoreError.duplicateID(event.id) }
            modelContext.insert(StoredEvent(event))
        }
        try modelContext.save()
    }

    /// All events in chronological order. Tombstones are filtered in the query
    /// unless requested.
    func allEvents(includeDeleted: Bool = false) throws -> [FoldEvent] {
        var descriptor = FetchDescriptor<StoredEvent>(sortBy: [SortDescriptor(\.timestamp)])
        if !includeDeleted {
            descriptor.predicate = #Predicate { $0.deletedAt == nil }
        }
        var zones: [String: TimeZone] = [:]
        return try modelContext.fetch(descriptor).compactMap { $0.domainEvent(zones: &zones) }
    }

    /// Everything the UI needs, computed here — off the main thread — in one pass.
    func loadState(now: Date, timeZone: TimeZone, recentLimit: Int = 500) throws -> LoadedState {
        let events = try allEvents()
        return LoadedState(dashboard: DashboardStats.make(from: events, now: now, timeZone: timeZone),
                           recentEvents: Array(events.filter(\.isLive).suffix(recentLimit).reversed()))
    }

    func count() throws -> Int {
        try modelContext.fetchCount(FetchDescriptor<StoredEvent>())
    }

    /// Replaces an existing event's fields and bumps `updatedAt`.
    func update(_ event: FoldEvent, now: Date = .now) throws {
```

```
        let row = try find(event.id)
        var changed = event
        changed.updatedAt = now
        row.apply(changed)
        try modelContext.save()
    }

    /// Tombstones an event (never physically removed, so imports can't resurrect it).
    func softDelete(id: UUID, now: Date = .now) throws {
        let row = try find(id)
        row.deletedAt = now
        row.updatedAt = now
        try modelContext.save()
    }

    /// Erases everything (Settings → Reset all data).
    func deleteAll() throws {
        try modelContext.delete(model: StoredEvent.self)
        try modelContext.save()
    }

    private func find(_ id: UUID) throws -> StoredEvent {
```

Packages/FoldPulseCore/Sources/FoldPulseCore/Statistics.swift

```
    public static func dailyAverage(_ events: [FoldEvent], today: LocalDay, window: Int = 7, includeInferred: Bool = false) -> Double?
{
        guard window > 0, let first = firstTrackedDay(events) else { return nil }
        let windowStart = max(today.adding(days: -window), first)
        let days = LocalDay.range(from: windowStart, through: today.adding(days: -1))
        guard !days.isEmpty else { return nil }
        let sum = series(events, days: days, includeInferred: includeInferred).reduce(0) { $0 + $1.count }
        return Double(sum) / Double(days.count)
    }

    /// The day with the most folds among `days` (earliest wins a tie). Nil if all are zero.
    public static func busiestDay(_ events: [FoldEvent], days: [LocalDay], includeInferred: Bool = false) -> DayCount? {
        let counts = series(events, days: days, includeInferred: includeInferred)
```

5 Build & test verification

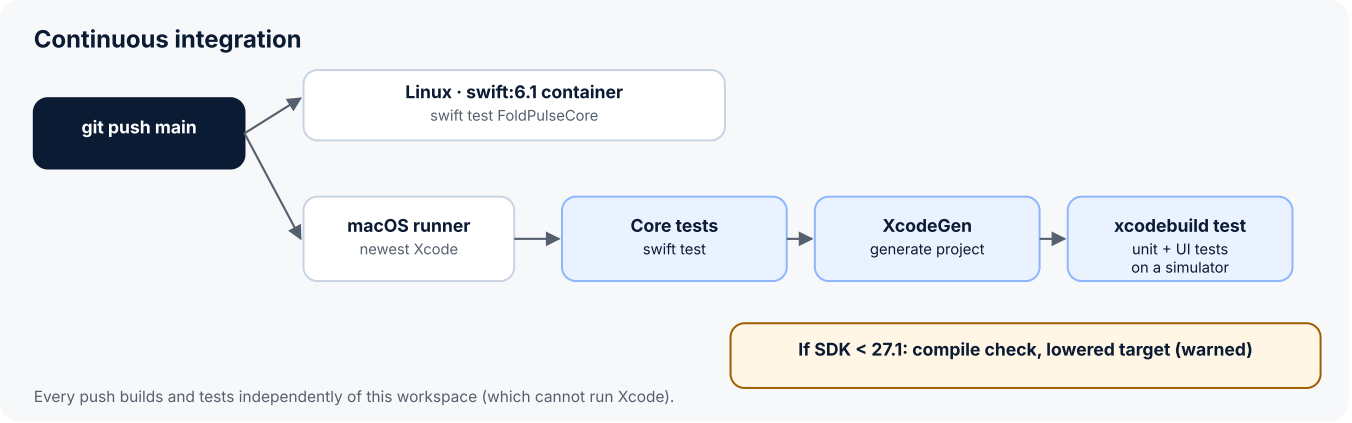


Figure 4 — Continuous integration

1. Result

Green. The final verified build is commit `a154fc1` (run #5, https://github.com/odthientho/FoldPulse/actions/runs/42703454967), with the same results as run #4 (`825507e`) and **no build warnings** apart from the expected App Intents note. Every job and step passed:

Job / step	Environment	Result
Core tests (Linux)	`swift:6.1-noble` container, x86_64	**29 tests, 0 failures**
Core tests (macOS)	Xcode 26.6, arm64	**29 tests, 0 failures**
Privacy guard (no networking symbols in shipping Swift code)	macOS	**Pass**
Generate Xcode project (XcodeGen 2.46.0)	macOS	**Pass**
Build app + widget extension, run unit tests	iPhone 17 Pro simulator, iOS 26.5 SDK	**13 tests, 0 failures**
UI test (launch → Today empty state → all four tabs)	Same simulator	**1 test, 0 failures** (20.9 s)
Overall		`** TEST SUCCEEDED **`

2. Test inventory

Suite	Tests	What it proves
`LocalDayTests`	7	Parsing, travel across zones, DST (23-hour day, repeated fall-back hour), month/year/leap boundaries, ranges, coding
`StatisticsTests`	8	Totals exclude gaps, tombstones and duplicates; capture day survives travel; full-day averages with zero days and a 7-day cap; series; busiest day; coverage
`MilestonesTests`	2	Next milestone and progress; each milestone is celebrated once, even after deletions
`WidgetSnapshotTests`	4	Snapshot values, encoding round trip, rejects garbage or newer schema, midnight rule, staleness
`ReviewFixTests`	8	Regressions from the agent reviews: impossible dates, timestamp edits recompute the day, stored day reused, unknown-zone fallback, skipped midnight, inferred first day, `includeInferred` pass-through, dashboard week
`EventStoreTests` (app)	10	SwiftData round trip of every field, ordering, update, tombstone, not-found, duplicate id rejected, batch add, unknown rows skipped, duplicates hidden from recent events, delete-all
`SnapshotServiceTests` (app)	3	Snapshot written and readable; background reloads coalesced across processes; protected store directory created
`LaunchTests` (UI)	1	App launches into Today (empty state) with all tabs reachable

Suite	Tests	What it proves
Total	**43**	

3. Problems CI found, and fixes

Run	Problem	Cause	Fix
#1	Cancelled	A second push superseded it (concurrency setting)	Pushes now batched while a run is in flight
#2	iOS job: "Unable to find a destination"	Runner has the **iOS 26.5 SDK**; the deployment-target override on the command line didn't apply when matching the destination	CI lowers the target in `project.yml` before generating (CI only)
#3	1 of 29 core tests failed	Test asserted the zone *name* "UTC"; Foundation reports "GMT" on both platforms	Test checks the zero offset; code uses one explicit `fallbackTimeZone`
#4	Green; build warning: widget version 1.0 ≠ app 0.1.0	XcodeGen default for the extension	Extension version follows the app's settings (commit `a154fc1`)
#5	**Green, warning gone**	—	—

No compile errors occurred in the app, widget or test code, under **Swift 6 with complete strict concurrency**.

4. What this verification does not cover

Gap	Why	Covered by
iOS 27.1 SDK / iPhone Duo simulator	GitHub's runners have Xcode 26.6 (iOS 26.5 SDK) at most	Founder's Mac with Xcode 27.1 (one-line setup script), plus CI once runners update
Physical-device behaviour (file protection while locked, widgets on the Lock Screen and in StandBy, App Group signing)	Needs hardware	Phase 5 device tests (spike criteria S-1…S-7)
Hinge, Shortcuts and App Intents	Phase 4 features	Phase 4
Remaining warning: "AppIntents metadata extraction skipped"	No App Intents exist yet	Disappears in Phase 4

6 Agent reviews & resolutions

How the reviews were run

Two independent reviewer subagents read the complete Phase 3 code and documentation. They could not edit anything. Neither could compile, so CI checked compilation separately.

Reviewer	Roles
A	Agent 6 — Senior iOS Software Engineer; Agent 11 — Performance & Battery Engineer
B	Agent 8 — Privacy & Security; Agent 9 — QA & Automation; Agent 12 — Independent Final Reviewer

Both returned **approve with changes**. Every High and Medium finding was fixed in commit `4185837` before this report. Low findings were fixed or are scheduled as listed.

Engineer (Agent 6)

Finding	Sev.	Resolution
Throttled background widget reloads were dropped and never retried; the throttle lived only in memory, so it reset in every short-lived intent process	High	Last-reload time stored in App Group defaults (tested across two instances); the widget re-reads the snapshot every 30 min
"as of HH:mm" never appeared: no timeline entry at the stale point	Medium	Entry added at `generatedAt + 15 min`
Editing `timestamp` left `localDay` on the old day	Medium	`timestamp` / `localDay` are `private(set)`; `setTimestamp` recomputes in the capture zone (tested)
`add` with an existing id silently overwrote the row (`.unique` upsert)	Medium	`EventStore.add` rejects duplicates (tested)
UI tests overwrote the real widget snapshot and reloaded widgets	Medium	UI-test mode uses a temp file and no reload
Release builds silently fell back from the App Group	Low	Debug-only fallback; release shows a clear error
`LocalDay(string:)` accepted 2026-02-31	Low	Validated with leap-year arithmetic (tested)
History showed merged duplicates	Low	`recentEvents` contains live events only (tested)

Performance & Battery (Agent 11)

Finding	Sev.	Resolution
Statistics recomputed on the main thread inside the view body (about 7 full passes per render), contradicting the docs	High	`DashboardStats` computed once per reload inside the `EventStore` actor; views only read it
Per-row `Calendar` and `TimeZone` work while loading; tombstones filtered in memory	High	Stored `localDay` reused, time-zone cache, `#Predicate { deletedAt == nil }`
Double reload at launch (and on every foreground)	Medium	`.task` at launch; reload on return from background only; midnight rollover via `NSCalendarDayChanged`
`@ModelActor` created on the main thread may run its context there	Medium	Created on a detached task
Sample-data insert saved once per event	Low	Batch `add(contentsOf:)` with one save
Widget memory	—	Fine: reads a small JSON file only

Privacy & Security (Agent 8)

Finding	Sev.	Resolution
App Group fallback could strand data	Medium	Fixed (see Engineer)
Store file protection assumed, not set	Medium	Store directory created with `completeUntilFirstUserAuthentication`
Privacy manifest declared an unused reason (CA92.1)	Low	App: `1C8F.1` (actual App Group defaults use); widget: none

Finding	Sev.	Resolution
`-ui-testing` honoured in release	Low	Debug-only
Promised "CI check for networking symbols" didn't exist	Low	CI privacy-guard step added

QA & Automation (Agent 9)

Finding	Sev.	Resolution
Missing tests: unknown enum rows, unknown time zone, skipped midnight, inferred first event, `includeInferred` pass-through	Medium	All added
Migration fixture test	Medium	**Deferred to the first schema change (V2)**: there is no older schema to migrate from yet. Recorded in DATABASE_DESIGN §6
Empty simulator UDID gave an unclear error	Low	Explicit failure
SDK-fallback run passed with only a warning	Low	Job summary now states "compile check on iOS 26.5 SDK"
CI does fail on test failures	—	Confirmed
FoldPulseCore not in Xcode coverage; XcodeGen not pinned	Low	Core coverage comes from `swift test`. Pinning is scheduled for Phase 4

Independent Final Reviewer (Agent 12)

Finding	Sev.	Resolution
Docs overclaimed: ids "enforced by the store", reset "removes files", protection, PRIVACY_REVIEW not updated	Medium	DATABASE_DESIGN, ARCHITECTURE and PRIVACY_REVIEW corrected (v1.1)
"Verified build" claimed before CI passed	Low	The build-verification chapter uses actual CI results and links
Coverage of the Phase 3 brief	—	All items present: architecture, project, folders, 4 targets, widget extension, data model, persistence, design system, components, tests

Disagreements

None between the reviewers in this phase. One limitation is noted: CI compiles against the **iOS 26.5 SDK**, because GitHub's runners don't yet have Xcode 27.1. The 27.1 build is verified on the Founder's Mac.

7 Technical decisions

Product and technical decisions, newest first. Only the Founder makes product decisions. Changes to an approved decision require a new entry.

ID	Date	Decision	Decided by	Context / consequence
A-2	2026-11-09	**Phase 2 approved** (design system, icon, screens, flows, widgets)	Founder	Open items carried: R-28 (verify background counting before pricing), I-24 (inner tab bar when partially folded → check in Phase 3)
C-2r	2026-11-09	**Skip the remaining device tests; continue to Phase 2**	Founder	Foreground hinge counting was confirmed informally (5/5). Background automation is unverified (the trigger wasn't found). Background counting is designed as optional; S-1…S-7 become a Phase 5 gate. See SPIKE_RESULTS.md
A-1	2026-11-09	**Phase 1 approved** (feasibility report + documents)	Founder	Recommendation CONDITIONAL GO accepted
C-2	2026-11-09	**Run the device validation spike (Phase 1b) before Phase 2**	Founder	Phase 2 starts after spike results are reviewed. Kit: `docs/SPIKE_PLAN.md`, `spike/`
C-1	2026-11-09	Founder provides a **Mac with Xcode 27.1+ and an iPhone Duo**	Founder	Resolves R-01 for the spike. CI for later phases is still to be decided in Phase 3
D-5	2026-11-09	**Business model: paid up front** (one-time download price, no in-app purchases)	Founder	Design needs no paywall, tip or upgrade surfaces. StoreKit is not needed. Pricing is set in Phase 6. New risk R-26 (paid vs. free competitors)
D-1	2026-11-09	**Build FoldPulse despite existing fold counters**; compete on trust (coverage indicator), widgets and polish	Founder (accepted team default)	R-05/R-21 remain as tracked risks
D-2	2026-11-09	**Inferred (gap) events excluded** from headline totals by default	Founder (default)	Shown in History with explanation
D-3	2026-11-09	**Minimum iOS 27.1**, iPhone Duo-focused	Founder (default)	Background assisted mode needs iOS 27.2+ on the device
D-4	2026-11-09	**Manual-only mode with a notice** on non-hinge iPhones and iPads	Founder (default)	Addresses App Review 2.4.1
D-6	2026-11-09	**Lean v1** (MVP_SCOPE §4.1)	Founder (default)	Medium/Large/inline widgets, heat strip, session stats, inferred toggle and JSON import (pending backup check) move to v1.1
I-17	2026-11-09	**Move FoldPulse to its own GitHub repository**	Founder (default)	Done: `odthientho/FoldPulse` (`main`). Repo is currently **public**; the Founder decides on visibility
T-4	2026-11-09	**XcodeGen** (`project.yml`) generates the Xcode project; `.xcodeproj` is not committed	Architect (Phase 3)	Reproducible project from text; easy review; no merge conflicts in pbxproj
T-5	2026-11-09	**FoldPulseCore** Swift package holds all domain logic (pure Swift)	Architect (Phase 3)	Deterministic, testable on Linux CI; app/widget share one implementation
T-6	2026-11-09	**SwiftData confirmed** over Core Data (DATABASE_DESIGN §1)	Architect (Phase 3)	`@ModelActor` single writer, versioned schema; Core Data remains fallback
T-7	2026-11-09	**Swift 6, complete strict concurrency** in all targets	Architect (Phase 3)	Data-race safety for background intents + widget + UI
T-8	2026-11-09	**GitHub Actions CI** (Linux core tests + macOS simulator build/tests); public repo → no cost	Architect (Phase 3)	The cloud workspace cannot run Xcode; CI provides independent build verification
T-9	2026-11-09	Bundle IDs `io.github.odthientho.foldpulse[.widgets]`, App Group `group.io.github.odthientho.foldpulse`	Architect (Phase 3)	Unique, owner-based; changeable before first App Store submission
T-1	2026-11-09	Event log is the source of truth; aggregates are computed/cached	Architect (Phase 1)	See TECHNICAL_FEASIBILITY §6–§7

ID	Date	Decision	Decided by	Context / consequence
T-2	2026-11-09	SwiftData in the App Group, single writer in the app process; widget writes via a queue file	Architect (Phase 1)	Confirmed or replaced in Phase 3
T-3	2026-11-09	No networking code, no third-party SDKs	Privacy (Phase 1)	PRIVACY_REVIEW P-1

8 Decisions requested & next steps

Decisions requested

#	Decision	Team recommendation
A-3	**Approve Phase 3** (architecture, project foundation, data layer, design system in code, tests and CI)	Approve
R-29	Apple Developer Program membership ($99/year): needed for App Store release, and probably for App Groups on your device	Join before Phase 4 device testing, or at the latest before Phase 6
R-28	Run the 10-minute background-automation check on iOS 27.2 before pricing	Strongly recommended (carried over from Phase 2)

Optional now: try the foundation on your iPhone Duo

1. Run the one-line setup in Terminal: `curl -fsSL https://raw.githubusercontent.com/odthientho/FoldPulse/main/scripts/setup_mac.sh | bash`
2. In Xcode, choose your Team for **FoldPulse** and **FoldPulseWidgets**, select your iPhone Duo, and press ▶.
3. In the app: **Settings → Insert sample week**. Today, History and the widget then show the design with sample data.
4. **I-24:** also check the inner-display tab bar with the phone half folded.

Phase 4 — MVP development (after approval)

Features are built one at a time, each with a plan, implementation, tests, CI and review:

1. **Event pipeline** in FoldPulseCore: debounce, baseline-on-activation, symmetric de-duplication (±10 s) and gap reconciliation, all unit-tested on Linux.
2. **Automatic counting:** one device-level `onHingeChange` observer.
3. **Manual recording** with Undo.
4. **App Intents** (Record Fold / Unfold, background) and the guided Shortcuts setup with self-test.
5. **History:** filters, edit, delete.
6. **Insights:** D/W/M/Y, averages, busiest day.
7. **Coverage** detail.
8. **Milestones**, celebrated once.
9. **Widgets** final polish.
10. **Settings:** export (CSV/JSON) and reset.
11. **Onboarding**, manual-only mode, accessibility pass.

Deliverables: `phase-04-development.pdf`, `DEVELOPMENT_LOG.md`, an updated ARCHITECTURE.md and test results.